Printer thanks joe

Spruce version 11.2 -- spooler version 11.2

File: DSK: KMP; EI 3

Creation date: 20 DEC 1984 1413

For: KMP

6 total sheets = 5 pages, 1 copy.

Need some mechanism for defining simple and compound conditions.

eg,

```
        (DEFINE-SIMPLE-CONDITION (name self-var (var1 default1)
                                                (var2 default2)
                                                ...)
          . default-action)

        (DEFINE-COMPOUND-CONDITION name
          . names-composed-of)
```

as in

```
        (DEFINE-SIMPLE-CONDITION (ERROR SELF
                                  (FORMAT-STRING "An error has occurred.")
                                  (FORMAT-ARGS '()))
          (DEBUGGER (TEXT FORMAT-STRING FORMAT-ARGS)))
```

NOW create macro DEFINE-ERROR-CONDITION which is like DEFINE-SIMPLE-CONDITION
to some internal name plus a join to ERROR.

```
(DEFINE-ERROR-CONDITION (JOE SELF (STUFF NIL))
  (PRINT (LIST JOE (JOE-STUFF SELF))))
```

Do we want to do this instead?

```
(LET ((STUFF (JOE-STUFF SELF))
      (...))
  ...)
```

so the guy can reference STUFF in the body?

```
(DEFINE-ERROR-CONDITION (JOE SELF (STUFF NIL))
  (PRINT (LIST JOE STUFF)))
```

==>

```
(DEFINE-SIMPLE-CONDITION (SIMPLE-JOE SELF (STUFF NIL))
  (PRINT (LIST JOE STUFF)))

(DEFINE-COMPOUND-CONDITION JOE (SIMPLE-JOE ERROR)
  . action)
```

There is a subtle difference between

```
 (SIGNAL (JOIN SIMPLE-JOE ERROR))
 (SIGNAL JOE)
```

```
;;; -*- Mode:LISP -*-

>>> (DEFINE (NEGATE X)
      (CHECK-ARG NUMBER? X)
      (- X))

==> (DEFINE (NEGATE X)
      (IF (NOT (NUMBER? X))
          (ERROR WRONG-TYPE-ARGUMENT
                 'VARIABLE-NAME 'X
                 'VARIABLE-VALUE X
                 'PROCEED-CASES
                   (PROCEED-CASE-CLUSTER
                    ((USE-VALUE V) (SETQ X V))
                    ((USE-ANY-VALUE) (SETQ X (RANDOM))))))
      (- X))

==> (DEFINE (NEGATE X)
      (IF (NOT (NUMBER? X))
          (ERROR WRONG-TYPE-ARGUMENT
                 'VARIABLE-NAME 'X
                 'VARIABLE-VALUE X
                 'PROCEED-CASES
                   (OBJECT NIL
                     ((VALID-PROCEED-TYPE? IGNORED-SELF G0001)
                      (OR (EQ? G0001 USE-VALUE)
                          (EQ? G0001 USE-ANY-VALUE)))
                     ((USE-VALUE IGNORED-SELF V G0001)
                      (SETQ X V))
                     ((USE-ANY-VALUE IGNORED-SELF G0001) (SETQ X (RANDOM))))))
      (- X))
```

Need to send messages to the USE-VALUE or USE-ANY-VALUE messages
in order to ask what they want to do about prompting. This is analagous
to the :CASE :PROCEED methods of LispM error flavors, but the thing is
that the proceed methods are not a property of the message condition at
all. eg,

```
>>> (DEFINE-PROCEED-CASE (USE-ANY-VALUE CONDITION))

    (DEFINE-PROCEED-CASE (USE-VALUE CONDITION)
      (THE-VALUE (PROMPT-AND-READ NUMBER?
                                  "~%Use what value for ~S instead of ~S: "
                                  (VARIABLE-NAME  CONDITION)
                                  (VARIABLE-VALUE CONDITION))))

    (CONDITION-BIND ((UNBOUND-VARIABLE?
                       (LAMBDA (CONDITION)
                         (PROCEED CONDITION USE-ANY-VALUE)))
                     (UNDEFINED-FUNCTION?
                       (LAMBDA (CONDITION)
                         (PROCEED CONDITION USE-FUNCTION *SOME-FUNCTION*))))
      (EVAL FORM))

==> (DEFINE-OPERATION (USE-ANY-VALUE VALUE-TO-USE)
      ((PROMPT IGNORED-SELF CONDITION G0001)
       (LET* () (G0001))))

    (DEFINE-OPERATION (USE-VALUE VALUE-TO-USE)
      ((PROMPT IGNORED-SELF CONDITION G0001)
       (LET* ((THE-VALUE (PROMPT-AND-READ
                           NUMBER?
                           "~%Use what value for ~S instead of ~S: "
                           (VARIABLE-NAME  CONDITION)
                           (VARIABLE-VALUE CONDITION))))
         (G0001 THE-VALUE))))

    (DEFINE (PROCEED CONDITION PROCEED-TYPE . PROCEED-ARGUMENTS)
      (LET ((CASES (PROCEED-CASES CONDITION)))
        (UNLESS (VALID-PROCEED-CASE? CASES CONDITION)
          (ERROR "~S is not a valid proceed type for ~S"
                 PROCEED-TYPE CONDITION))
        (APPLY PROCEED-TYPE CASES PROCEED-ARGUMENTS)
        (INTERNAL-RETURN-FROM-ERROR)))

    (DEFINE (PROMPTED-PROCEED CONDITION PROCEED-TYPE)
      (PROMPT PROCEED-TYPE CONDITION
              (LAMBDA ARGS
                (APPLY PROCEED CONDITION PROCEED-TYPE ARGS))))
```

This should pretty much describe a non-value-producing error recovery system.

```
(DEFINE (ERROR TYPE . KEYWORDED-ARGS)
  (LET ((ERROR-OBJECT (APPLY [maker] TYPE KEYWORDED-ARGS)))
    (LET HANDLER-LOOKUP ((HANDLERS (FLUID-HANDLER-STATE)))
         (IF (NULL? HANDLERS)
             (HANDLE-BY-DEFAULT ERROR-OBJECT)
           (LET ((CLAUSE1 (FIRST HANDLERS))
                 (CLAUSES (REST  HANDLERS)))
             (IF ((FIRST CLAUSE1) ERROR-OBJECT)
                 (BIND ;ick
                       ((DISMISS (LAMBDA () (HANDLE-BY-DEFAULT CLAUSES))))
                   (SECOND CLAUSE1))
               (HANDLE-BY-DEFAULT CLAUSES)))))))
```

Where are all the type predicates coming about?

Where is distinction between simple and complex types?

```
;;; Local Modes:
;;; Lisp CONDITION-BIND Indent:1
;;; Lisp OBJECT Indent:1
;;; End:
```